

Yacoub Projekt plan

Projektplan - Infrastrukturoptimering och prestandaanalys av Agentic AI i Edge-miljöer

Student: Yacoub Dawli

1. Bakgrund och Problem beskrivning

Driftsättning av Agentic AI (autonoma system som kan planera och använda verktyg) kräver vanligtvis betydande molnresurser, att köra dessa arbetsbelastningar på Edge-enheter (som en Raspberry Pi) innebär stora begränsningar gällande minne (ram), cpu cykler och thermal throttling(1).

Enligt Sapkota(2) skiljer sig Agentic AI från traditionella modeller genom sin förmåga att agera autonomt. Detta kräver dock betydande beräkningskraft. Zhou (3) belyser att flytt av sådan intelligens till Edge-enheter (Edge Intelligence) minskar nätverkslatens, men introducerar utmaningar gällande hårdvarans begränsade resurser.

Problemet är att standardverktyg för orkestrering av n8n är optimerade för servrar, inte för begränsade IoT-enheter. Utan specifik optimering kommer "kroppen" (hårdvara/operativsystem) att krascha när "hjärnan" (AI-agenten) försöker utföra komplexa resonemangsuppgifter (4).

2. Syfte och mål

Syftet med detta examensarbete är att designa, implementera och stresstesta en resurseffektiv infrastruktur med låg latens för drift av en autonom AI-agent.

Mitt specifika mål är att kvantifiera de operativa begränsningarna hos en Raspberry Pi när den agerar värd för en n8n-baserad AI-agent, samt att bygga de gränssnitt för mjuk- och hårdvara som krävs för att AI-agenten ska kunna interagera med den fysiska världen.

3. Omfattning och ansvarsområden (Yacoub)

Mitt arbete är uppdelat i tre huvudsakliga ingenjörsmässiga områden

A. Miljön/Containern

- Uppgift - För att hantera dessa hårdvaru begränsningar används container-teknik. Morabito (5) visar att Docker på IoT-enheter kan introducera prestandaförluster om det inte konfigureras korrekt. Därför syftar detta arbete till att optimera container-miljön ... att Implementera och optimera en självhostad n8n-instans på Raspberry Pi med hjälp av Docker.
- Optimering - Gå bortom standardkonfigurationen för att säkerställa stabilitet.
Detta innefattar:
 - Begränsning av containerns minnesanvändning (RAM) för att förhindra systemlåsning
 - Konfigurering av n8ns exekveringsläge med tunneling kontra lokal, för att maximera hastigheten
 - Val av lättviktiga os (tex Alpine Linux (6) istället av Debian)

B. Gränssnittet

- Uppgift - Skapa en brygga mellan de fysiska sensorerna och det digitala arbetsflödet. Eftersom AI (Obids del) inte fysiskt kan "se" värme eller "röra" en fläkt, måste den interagera via API anrop.

- Leverans - Utveckling av ett middleware i Python.
 - Input - Skript som läser GPIO-data (temperatur, rörelse) och skickar detta till n8n via Webhooks (hårdvaran pushar endast ett meddelande när någonting faktiskt händer(7))
 - Output - API-endpoints som AI-agenten kan anropa för att trigga fysiska åtgärder (t.ex `OST /fan/on`)

C. Mätvärden

- Uppgift - Bygga ett övervakningssystem för att mäta "kostnaden" för intelligens på hårdvaran.
- Leverans - Automatiserad loggning av
 - Systemlatens (tiden mellan sensortriggning → utförande av åtgärd.
 - Thermal prestanda - cpu temperatur i relation till agentens komplexitet.
 - Resursbelastning: Spikar i ram och cpu användning under AI inferens.

4. Tidsplan

Arbetet beräknas totalt omfatta ca 350 timmar och är fördelat över vårterminen (februari – juni) med en ökande arbetsintensitet inför genomförandefasen.

Total arbetsfördelning (350 timmar):

April Sätta upp den tekniska grunden: självhostad n8n på Raspberry Pi/Docker, grundläggande containeroptimering samt den första lokala testmiljön. Påbörja Python-middleware och definiera den första API-kopplingen mellan sensorer och n8n.

Maj Implementera bryggan mellan hårdvara och mjukvara i praktiken: GPIO-avläsning från sensorer, webhook-kommunikation in till n8n samt API-endpoints

för åtgärder som `/fan/on`. Samtidigt införs övervakning av latens, CPU, RAM och termisk prestanda.

Juni Slutföra integration och utvärdering. Genomföra upprepade tester av hela flödet från sensor till åtgärd, analysera Raspberry Pi:ns operativa begränsningar och sammanfatta den uppmätta prestandan för den edge-baserade infrastrukturen.

References

1. Agentic AI

[1] S. Hosseini and H. Seilani, "The role of agentic AI in shaping a smart future: A systematic review," *Art. no. 100399*, May 2025.

[2] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges," *arXiv preprint arXiv:2505.10468*, May 2025.

2. Edge Computing och Edge Intelligence

[3] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," in *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[4] M. Merenda, C. Porcaro, and D. Iero, "Edge Machine Learning for IoT: A Novel Alliance," *Sensors*, vol. 20, no. 9, p. 2548, May 2020.

3. Virtualisering och Docker för IoT

[5] R. Morabito, "Virtualization on Internet of Things: Edge computing with containers and virtual machines," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 1707–1719, Oct. 2017.

4. Teknisk dokumentation & Böcker

[6] **Alpine Linux (u.å.).** About Alpine Linux. [Online] Tillgänglig på: <https://www.alpinelinux.org/about/> [Hämtad: 2026-02-12].§§

[7] S. Greengard, *The Internet of Things*, updated ed. Cambridge, MA, USA: MIT Press, 2021.